Building a Windows Computer-Use MCP Server in C# with Native AOT — Research & Implementation Guide

TL;DR

- **Build on the official `ModelContextProtocol` C# SDK (Microsoft/Anthropic, stable v1.3.0 on NuGet as of May 2026 — the SDK shipped its 1.0.0 release the week of February 23, 2026)** — it is explicitly Native-AOT compatible (PR #1246 added an AOT publish test to CI), and a full Windows-control server with multi-monitor screenshot, mouse, keyboard, and file ops ships as a single ~15 MB `.exe` when published with `<PublishAot>true</PublishAot>`.
- **Model your tool surface on Anthropic's `computer_20250124` action set augmented with Windows-MCP's coarser tools** — screenshot, mouse_move/click/drag/scroll, key/type/hotkey, plus monitor enumeration and file I/O. Adopt Anthropic's exact downscale-then-remap pipeline from `claude-quickstarts/computer-use-demo/computer_use_demo/tools/computer.py`: capture at native resolution, downscale to XGA/WXGA/FWXGA, persist the scale factor + monitor origin, then map AI-returned coordinates back with `x_real = round(x_model / x_factor) + monitor.Left`.
- **Use Win32 P/Invoke via `LibraryImport` (source-generated, AOT-safe) for everything OS-level**: `EnumDisplayMonitors`/`GetMonitorInfo`/`GetDpiForMonitor` for displays, `BitBlt` for capture, `SendInput` with `MOUSEEVENTF_VIRTUALDESK | MOUSEEVENTF_ABSOLUTE` for mouse, and `KEYEVENTF_UNICODE` for typing. Use **SixLabors.ImageSharp 4.0** (released May 12, 2026, targets .NET 8+) for PNG encoding, resize, and grayscale — it publishes cleanly with `PublishAot=true` since v3.1.0's `Configuration.Default` fix (issue #2486). Avoid `System.Drawing.Common`: it is in maintenance-only mode and pulls a GDI+ native dependency. Sixlabors

---

Key Findings

1. The C# MCP SDK is Native-AOT-ready, but tool registration must avoid reflection

The official `modelcontextprotocol/csharp-sdk` (maintained jointly with Microsoft, packages `ModelContextProtocol.Core`, `ModelContextProtocol` 1.3.0, `ModelContextProtocol.AspNetCore` 1.3.0) added an "AOT compatibility publish test #1246" (stephentoub) and an SSE example built with `<PublishAot>true</PublishAot>` produces a ~15.7 MB single-file Windows `.exe` (Laurent Kempé, "SSE-Powered MCP Server with C# and .NET in 15.7MB executable"). The standard `[McpServerToolType]` / `[McpServerTool]` attributes plus `.WithToolsFromAssembly()` work, but `WithToolsFromAssembly()` emits an IL2026 trim warning under AOT (csharp-sdk issue #317) because it scans the assembly via reflection. The supported AOT-clean path is to register each tool type explicitly via `.WithTools<MyToolsClass>()`, and to pass a `JsonSerializerOptions` configured with a

`[JsonSerializable]` source-generated `JsonSerializerContext` so all parameter and return types are statically rooted. GitHub + 4

2. Anthropic's "computer" tool is the canonical action model

From `anthropics/claude-quickstarts/computer-use-demo/computer_use_demo/tools/computer.py`, the action enums are:

- `Action_20241022`: `key`, `type`, `mouse_move`, `left_click`, `left_click_drag`, `right_click`, `middle_click`, `double_click`, `screenshot`, `cursor_position` GitHub github
- `Action_20250124` adds: `left_mouse_down`, `left_mouse_up`, `scroll`, `hold_key`, `wait`, `triple_click` GitHub github
- `Action_20251124` adds: `zoom` (region inspection at full resolution) GitHub + 2

The model expects pixel coordinates in the resolution declared in the tool definition (`display_width_px`, `display_height_px`). Anthropic explicitly recommends keeping that declared resolution at **XGA (1024×768) or WXGA (1280×800)** — the demo's `MAX_SCALING_TARGETS` dictionary contains exactly three named targets: `XGA` (1024×768, 4:3), `WXGA` (1280×800, 16:10), `FWXGA` (1366×768, ~16:9). From the demo's README: *"We do not recommend sending screenshots in resolutions above XGA/WXGA to avoid issues related to image resizing. Relying on the image resizing behavior in the API will result in lower model accuracy and slower performance than implementing scaling in your tools directly."* github GitHub

3. Windows-MCP defines the de-facto Windows tool surface

`CursorTouch/Windows-MCP` (2M+ Claude Desktop Extension users, MIT-licensed, Python) exposes a richer set than Anthropic's bare schema: **Click, Type, Clipboard, Scroll, Drag, Move, Shortcut, Key, Wait, State, Snapshot, Screenshot (with `display=[0]` or `display=[0,1]` for multi-monitor), Resize, Launch, Shell (PowerShell), Scrape, MultiSelect, MultiEdit, Process, Notification, Registry**. The `Screenshot` tool description explicitly supports per-monitor capture and applies a brief "orange-red glowing border" as a visual confirmation (disable via `WINDOWS_MCP_DISABLE_FLASH=1`). This is the right superset to mine for your feature list. GitHub + 3

4. P/Invoke and Native AOT compose cleanly; `LibraryImport` is the AOT-safe pattern

Microsoft Learn ("P/Invoke source generation"): *"Since [the DllImport] IL stub is generated at runtime, it isn't available for ahead-of-time (AOT) compiler or IL trimming scenarios. ... The P/Invoke source generator, included with the .NET 7 SDK and enabled by default, looks for LibraryImportAttribute on a static and partial method to trigger compile-time source generation of marshalling code."* Use `[LibraryImport("user32.dll", ...)]` on `static partial` methods for every Win32 call; `CharSet` becomes `StringMarshalling = StringMarshalling.Utf16`; for SetLastError use `SetLastError = true`. Direct P/Invoke is automatically generated by the AOT

compiler for `kernel32`, `user32`, `gdi32`, etc. listed in Native AOT's prepopulated Windows direct-PInvoke list. Microsoft Learn + 2

5. ImageSharp publishes under PublishAot; System.Drawing.Common does not

- **ImageSharp 3.1+ and 4.0** (4.0.0 released May 12, 2026, targets .NET 8+) publishes cleanly with `<PublishAot>true</PublishAot>` on Windows CoreCLR. Issue #2486 (filed by Michal Strehovsky, .NET runtime team) reported a Windows 11 / PublishAot binary-size problem; it was closed by PR #2514 in 3.1.0, whose release notes confirm: *"API changes have been implemented in the default Configuration API to address a problem that increased the size of AOT compiled assemblies by 77% more than anticipated (breaking)."* The library does not yet declare `IsAotCompatible=true` in its csproj — expect benign trim analyzer warnings — but the binary executes correctly. GitHub + 2
- `System.Drawing.Common` is Windows-only since .NET 6 and "will continue to evolve only in the context of Windows Forms and GDI+" (Microsoft Learn). It depends on native GDI+, ships its own native dependency on Linux (libgdiplus), and Microsoft now officially recommends "SkiaSharp and ImageSharp" as replacements. For a Native-AOT single-file Windows .exe you want the fully managed PNG/JPEG codec path that ImageSharp provides. Microsoft Learn GitHub

6. The downscale + coordinate-remap pipeline is the core of Anthropic's design

The reference implementation in `computer.py` from `anthropics/claude-quickstarts` is short and decisive — adopt the same algorithm verbatim, just in C#:

```python
```

```python
# verbatim from anthropics/claude-quickstarts/.../tools/computer.py
class ScalingSource(StrEnum):
    COMPUTER = "computer"
    API = "api"

MAX_SCALING_TARGETS: dict[str, Resolution] = {
    "XGA":  Resolution(width=1024, height=768),  # 4:3
    "WXGA": Resolution(width=1280, height=800),  # 16:10
    "FWXGA": Resolution(width=1366, height=768), # ~16:9
}

def scale_coordinates(self, source: ScalingSource, x: int, y: int):
    if not self._scaling_enabled:
        return x, y
    ratio = self.width / self.height
    target_dimension = None
    for dimension in MAX_SCALING_TARGETS.values():
        if abs(dimension["width"] / dimension["height"] - ratio) < 0.02:
            if dimension["width"] < self.width:
                target_dimension = dimension
                break
    if target_dimension is None:
        return x, y
    x_scaling_factor = target_dimension["width"]  / self.width
    y_scaling_factor = target_dimension["height"] / self.height
    if source == ScalingSource.API:
        if x > self.width or y > self.height:
            raise ToolError(f"Coordinates {x}, {y} are out of bounds")
        return round(x / x_scaling_factor), round(y / y_scaling_factor)   # scale up
    return round(x * x_scaling_factor), round(y * y_scaling_factor)        # scale d
```

Two non-obvious points:

- The reference scales only **down** (when target.width < screen.width). For lower-than-XGA screens, the README recommends *"Add black padding around the display area until it reaches 1024x768"* rather than upscaling. GitHub
- The aspect-ratio tolerance (`0.02`) avoids forcing a 16:9 native screen onto a 4:3 XGA target; pick the closest matching ratio.

For **multi-monitor** servers you add one step that Anthropic's Linux/Xvfb demo doesn't need: store the chosen monitor's virtual-desktop origin (`(monitor.Left, monitor.Top)`) alongside the scale factor and add it back after up-scaling.

7. Multi-monitor mouse coordinates have a well-known Windows gotcha

`MOUSEINPUT` with `MOUSEEVENTF_ABSOLUTE` alone maps `(0, 65535)` to the **primary monitor only** (Microsoft Learn). For multi-monitor systems you must OR in `MOUSEEVENTF_VIRTUALDESK`, and the (dx, dy) range then spans the entire virtual desktop, computed from `GetSystemMetrics` constants `SM_XVIRTUALSCREEN`, `SM_YVIRTUALSCREEN`, `SM_CXVIRTUALSCREEN`, `SM_CYVIRTUALSCREEN`. Filip Valentin's writeup concludes: *"You should always use MOUSEEVENTF_VIRTUALDESK when using MOUSEEVENTF_MOVE to simulate moving the mouse cursor on Windows."* filipvalentin@github

---

Details

A. Consolidated master tool list (mined from 6+ projects)

Tool name (suggested)	Source projects
`list_monitors`	(your requirement; not in Windows-MCP/Anthropic schema)
`screenshot`	Anthropic `screenshot`, Windows-MCP `Screenshot`, screenpipe
`mouse_move`	Anthropic, Windows-MCP `Move-Tool`
`mouse_click`	Anthropic `left_click`/`right_click`/`middle_click`/`double_click`/`triple_click` Windows-MCP `Click-Tool`
`mouse_down` / `mouse_up`	Anthropic `left_mouse_down`/`left_mouse_up`
`mouse_drag`	Anthropic `left_click_drag`, Windows-MCP `Drag-Tool`
`mouse_scroll`	Anthropic `scroll`, Windows-MCP `Scroll-Tool`
`key_press`	Anthropic `key`, Windows-MCP `Key-Tool`
`key_hotkey`	Windows-MCP `Shortcut-Tool`
`key_hold`	Anthropic `hold_key`

Tool name (suggested)	Source projects
`type_text`	Anthropic `type`, Windows-MCP `Type-Tool`
`cursor_position`	Anthropic
`wait`	Anthropic `wait`, Windows-MCP `Wait-Tool`
`clipboard_get` / `clipboard_set`	Windows-MCP `Clipboard-Tool`
`read_file`	(user requirement; mcp-essentials-windows)
`write_file`	(user requirement)
`create_folder`	(user requirement)
`launch_app`	Windows-MCP `Launch-Tool`
`shell`	Windows-MCP `Shell-Tool`

Anthropic's Windows-MCP also has `State-Tool` / `Snapshot-Tool` which return the UIA accessibility tree + interactive element coordinates; that's powerful but out of scope for a vision-first server. For coordinate-only operation matching the Anthropic computer_use pattern, the tools above are sufficient.

Comparison of language/library stacks across surveyed projects:

Project	Language	Input lib	Screen capture	Notes
anthropics/claude-quickstarts computer-use-demo	Python	shells out to `xdotool`	`gnome-screenshot`/`scrot` (Linux/Xvfb)	Reference for action schema + scaling
CursorTouch/Windows-MCP	Python (uv/uvx)	Windows UIA via `uiautomation`; mouse/keyboard via pyautogui-like layer	`mss`/PIL	Most popular Windows MCP
mukul975/mcp-windows-automation	Python	pyautogui, pywinauto	PIL	"200+ tools"
alxspiker/Windows-Command-Line-MCP-Server	TypeScript	none (shell only)	none	Command-execution only
mario-andreschak/mcp-essentials-windows	TypeScript	none	none	File + cmd + web fetch
screenpipe	Rust + TS (with terminator SDK)	OS-level "Playwright-for-desktop"	continuous screen+mic recording	Read-mostly; computer-use via separate "terminator" SDK
ModelContextProtocol.NET (salty-flower)	C#	n/a	n/a	Independent C# SDK explicitly marketed as "Native AOT Compatible" GitHub — predates the official Microsoft SDK; the official one is now preferred

No production-grade **C#/.NET** MCP server with the full computer-use surface exists yet on GitHub — this is a meaningful gap your project can fill.

B. .NET project skeleton

`MyWinComputerMcp.csproj`:

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <RuntimeIdentifier>win-x64</RuntimeIdentifier>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>

    <PublishAot>true</PublishAot>
    <OptimizationPreference>Size</OptimizationPreference>
    <InvariantGlobalization>true</InvariantGlobalization>
    <StripSymbols>true</StripSymbols>

    <!-- Surface IL3050/IL2026 early; treat as warning until ImageSharp ships IsAotC
    <IsAotCompatible>true</IsAotCompatible>
    <TrimmerSingleWarn>false</TrimmerSingleWarn>

    <ApplicationManifest>app.manifest</ApplicationManifest>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="ModelContextProtocol" Version="1.3.0" />
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="9.*" />
    <PackageReference Include="SixLabors.ImageSharp" Version="4.0.0" />
  </ItemGroup>
</Project>
```

`app.manifest` (declares Per-Monitor V2 DPI awareness so captured bitmap sizes and mouse coordinates are physical, not virtualized):

```xml
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <asmv3:application xmlns:asmv3="urn:schemas-microsoft-com:asm.v3">
    <asmv3:windowsSettings xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSetti
      <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">True/P
      <dpiAwareness xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">Pe
    </asmv3:windowsSettings>
  </asmv3:application>
</assembly>
```

This matches Marius Bancila's reference template ("How to build high DPI aware native Windows desktop applications"); without it, captures on 150%/200% scaled monitors come back blurry-upscaled and your mouse coordinates will be off. COM Apartments

`Program.cs`:

csharp

```csharp
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

var builder = Host.CreateApplicationBuilder(args);

// stdio: never log to stdout
builder.Logging.ClearProviders();
builder.Logging.AddConsole(o => o.LogToStandardErrorThreshold = LogLevel.Trace);

builder.Services
    .AddSingleton<MonitorRegistry>()        // caches EnumDisplayMonitors result + c
    .AddSingleton<CoordinateMapper>()
    .AddSingleton<ScreenCaptureService>()
    .AddSingleton<InputService>()
    .AddSingleton<FileService>()
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithTools<MonitorTools>()              // explicit registration, not WithToolsF
    .WithTools<ScreenTools>()
    .WithTools<MouseTools>()
    .WithTools<KeyboardTools>()
    .WithTools<FileTools>();

await builder.Build().RunAsync();
```

JSON source generation context — required to keep all parameter and return DTOs reflection-free under AOT:

```csharp
[JsonSerializable(typeof(MonitorInfo[]))]
[JsonSerializable(typeof(ScreenshotResult))]
[JsonSerializable(typeof(ClickRequest))]
[JsonSerializable(typeof(TypeTextRequest))]
[JsonSerializable(typeof(CoordSpace))]
[JsonSourceGenerationOptions(PropertyNamingPolicy = JsonKnownNamingPolicy.SnakeCaseL
internal partial class McpJsonContext : JsonSerializerContext { }
```

Pass `McpJsonContext.Default.Options` into `AddMcpServer().WithTools<T>(serializerOptions: ...)` — the SDK exposes overloads for this, and csharp-sdk issue #795 tracks ongoing work to make every `With*` method accept a `JsonSerializerOptions` so users can opt fully out of reflection-based schema generation. GitHub

C. Win32 P/Invoke layer (AOT-clean)

All declarations use `LibraryImport`. Example block (compresses what you need across the server):

```csharp
```

```
internal static partial class Win32
{
    // --- Monitors ---
    [LibraryImport("user32.dll")]
    [return: MarshalAs(UnmanagedType.Bool)]
    public static partial bool EnumDisplayMonitors(
        IntPtr hdc, IntPtr lprcClip, MonitorEnumProc lpfnEnum, IntPtr dwData);

    public delegate bool MonitorEnumProc(IntPtr hMon, IntPtr hdc, ref RECT rect, Int

    [LibraryImport("user32.dll", EntryPoint = "GetMonitorInfoW")]
    [return: MarshalAs(UnmanagedType.Bool)]
    public static partial bool GetMonitorInfo(IntPtr hMon, ref MONITORINFOEX info);

    [LibraryImport("shcore.dll")]
    public static partial int GetDpiForMonitor(IntPtr hMon, int dpiType, out uint dp

    // --- Virtual desktop metrics ---
    [LibraryImport("user32.dll")] public static partial int GetSystemMetrics(int idx
    public const int SM_XVIRTUALSCREEN = 76, SM_YVIRTUALSCREEN = 77,
                     SM_CXVIRTUALSCREEN = 78, SM_CYVIRTUALSCREEN = 79;

    // --- Screen capture (GDI) ---
    [LibraryImport("user32.dll")] public static partial IntPtr GetDC(IntPtr hwnd);
    [LibraryImport("user32.dll")] public static partial int ReleaseDC(IntPtr hwnd, I
    [LibraryImport("gdi32.dll")]  public static partial IntPtr CreateCompatibleDC(In
    [LibraryImport("gdi32.dll")]  public static partial IntPtr CreateCompatibleBitma
    [LibraryImport("gdi32.dll")]  public static partial IntPtr SelectObject(IntPtr h
    [LibraryImport("gdi32.dll")][return: MarshalAs(UnmanagedType.Bool)]
    public static partial bool BitBlt(IntPtr hdcDest, int x, int y, int cx, int cy,
                                      IntPtr hdcSrc, int sx, int sy, uint rop);
    public const uint SRCCOPY = 0x00CC0020, CAPTUREBLT = 0x40000000;
    [LibraryImport("gdi32.dll")] public static partial int GetDIBits(
        IntPtr hdc, IntPtr hbmp, uint start, uint cLines, IntPtr lpvBits,
        ref BITMAPINFO bmi, uint usage);
    [LibraryImport("gdi32.dll")] [return: MarshalAs(UnmanagedType.Bool)]
    public static partial bool DeleteObject(IntPtr o);
    [LibraryImport("gdi32.dll")] [return: MarshalAs(UnmanagedType.Bool)]
    public static partial bool DeleteDC(IntPtr dc);

    // --- Input ---
    [LibraryImport("user32.dll", SetLastError = true)]
    public static partial uint SendInput(uint nInputs,
        [In] INPUT[] pInputs, int cbSize);

    [LibraryImport("user32.dll")] [return: MarshalAs(UnmanagedType.Bool)]
```

```
    public static partial bool SetProcessDpiAwarenessContext(IntPtr ctx);
    public static readonly IntPtr DPI_AWARENESS_CONTEXT_PER_MONITOR_AWARE_V2 = new(-
}

[StructLayout(LayoutKind.Sequential)]
public struct RECT { public int Left, Top, Right, Bottom; }

[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Unicode)]
public struct MONITORINFOEX
{
    public int cbSize;
    public RECT rcMonitor, rcWork;
    public uint dwFlags;
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 32)] public string szDevice;
}

[StructLayout(LayoutKind.Sequential)]
public struct INPUT { public uint type; public InputUnion U; }
[StructLayout(LayoutKind.Explicit)]
public struct InputUnion
{
    [FieldOffset(0)] public MOUSEINPUT mi;
    [FieldOffset(0)] public KEYBDINPUT ki;
    [FieldOffset(0)] public HARDWAREINPUT hi;
}
[StructLayout(LayoutKind.Sequential)]
public struct MOUSEINPUT { public int dx, dy; public uint mouseData, dwFlags, time;
[StructLayout(LayoutKind.Sequential)]
public struct KEYBDINPUT  { public ushort wVk, wScan; public uint dwFlags, time; pub
```

`MOUSEINPUT.dwFlags` constants you need: `MOUSEEVENTF_MOVE=0x0001`, `LEFTDOWN=0x0002`, `LEFTUP=0x0004`, `RIGHTDOWN=0x0008`, `RIGHTUP=0x0010`, `MIDDLEDOWN=0x0020`, `MIDDLEUP=0x0040`, `WHEEL=0x0800`, `HWHEEL=0x01000`, `ABSOLUTE=0x8000`, `VIRTUALDESK=0x4000`. `KEYBDINPUT.dwFlags`: `EXTENDEDKEY=0x01`, `KEYUP=0x02`, `UNICODE=0x04`, `SCANCODE=0x08`.

D. Monitor enumeration + per-monitor DPI

csharp

```
public sealed class MonitorRegistry
{
    public IReadOnlyList<MonitorInfo> Monitors { get; private set; } = [];

    public MonitorRegistry()
    {
        Win32.SetProcessDpiAwarenessContext(Win32.DPI_AWARENESS_CONTEXT_PER_MONITOR_
        Refresh();
    }

    public void Refresh()
    {
        var list = new List<MonitorInfo>();
        int idx = 0;
        Win32.EnumDisplayMonitors(IntPtr.Zero, IntPtr.Zero, (IntPtr hMon, IntPtr _,
        {
            var mi = new MONITORINFOEX { cbSize = Marshal.SizeOf<MONITORINFOEX>() };
            Win32.GetMonitorInfo(hMon, ref mi);
            Win32.GetDpiForMonitor(hMon, 0 /*MDT_EFFECTIVE_DPI*/, out var dpiX, out
            list.Add(new MonitorInfo(
                Index: idx++,
                DeviceName: mi.szDevice,
                Bounds: new(mi.rcMonitor.Left, mi.rcMonitor.Top,
                            mi.rcMonitor.Right - mi.rcMonitor.Left,
                            mi.rcMonitor.Bottom - mi.rcMonitor.Top),
                WorkArea: new(mi.rcWork.Left, mi.rcWork.Top,
                              mi.rcWork.Right - mi.rcWork.Left,
                              mi.rcWork.Bottom - mi.rcWork.Top),
                IsPrimary: (mi.dwFlags & 1) != 0,
                DpiX: (int)dpiX, DpiY: (int)dpiY));
            return true;
        }, IntPtr.Zero);
        Monitors = list;
    }
}

public record MonitorInfo(int Index, string DeviceName, Rectangle Bounds,
                          Rectangle WorkArea, bool IsPrimary, int DpiX, int DpiY);
```

Expose a tool that returns this list as JSON. **This is the "ASK which monitor" step the user wants** — the host LLM calls `list_monitors` first, you return the array, the LLM (or the user) picks an index, and every subsequent screenshot/input call carries that `monitor_index`.

E. Screen capture — GDI BitBlt is the right default

Compared options:

- **GDI BitBlt** — simplest, AOT-safe (just user32/gdi32 P/Invokes), works for the entire virtual desktop or any sub-rectangle. Microsoft's "Multiple Monitor Applications" documentation states explicitly: *"The BitBlt function works well for multiple monitor systems."* Works in DWM-composited environments for most apps (excludes some DRM-protected windows). **Recommended.** Microsoft Learn
- **Windows.Graphics.Capture (WinRT)** — newer, supports hardware-accelerated captures and DRM-protected sources with consent prompts. Adds dependency on CsWinRT/WinRT-Interop; CsWinRT supports AOT but adds complexity. Defer unless you hit BitBlt limitations.
- **DXGI Desktop Duplication** — high-FPS continuous capture, overkill for on-demand screenshots; complicates AOT due to DirectX interop.

GDI capture of monitor index N:

```csharp
```

```
public byte[] CaptureMonitorPng(int monitorIndex, out int origW, out int origH,
                                out int monitorLeft, out int monitorTop)
{
    var m = _registry.Monitors[monitorIndex];
    monitorLeft = m.Bounds.X; monitorTop = m.Bounds.Y;
    origW = m.Bounds.Width;   origH = m.Bounds.Height;

    IntPtr screenDc = Win32.GetDC(IntPtr.Zero);
    IntPtr memDc = Win32.CreateCompatibleDC(screenDc);
    IntPtr bmp  = Win32.CreateCompatibleBitmap(screenDc, origW, origH);
    IntPtr old  = Win32.SelectObject(memDc, bmp);
    Win32.BitBlt(memDc, 0, 0, origW, origH, screenDc, m.Bounds.X, m.Bounds.Y,
                 Win32.SRCCOPY | Win32.CAPTUREBLT);

    // Pull raw BGRA via GetDIBits into a byte[], then hand to ImageSharp
    var pixels = ReadBGRA(memDc, bmp, origW, origH);

    Win32.SelectObject(memDc, old);
    Win32.DeleteObject(bmp);
    Win32.DeleteDC(memDc);
    Win32.ReleaseDC(IntPtr.Zero, screenDc);

    using var img = Image.LoadPixelData<Bgra32>(pixels, origW, origH);
    using var ms = new MemoryStream();
    img.Save(ms, new PngEncoder { CompressionLevel = PngCompressionLevel.BestSpeed });
    return ms.ToArray();
}
```

F. Image optimization + coordinate-mapping pipeline (C# port of Anthropic's logic)

csharp

```
public static class ScalingTargets
{
    public record Target(string Name, int Width, int Height);
    public static readonly Target[] All =
    [
        new("XGA",   1024, 768),  // 4:3
        new("WXGA",  1280, 800),  // 16:10
        new("FWXGA", 1366, 768),  // ~16:9
    ];
}

public readonly record struct ScalePlan(
    int OrigWidth, int OrigHeight,
    int ScaledWidth, int ScaledHeight,
    double FactorX, double FactorY,    // factor = scaled / orig, < 1
    int MonitorLeft, int MonitorTop);

public sealed class CoordinateMapper
{
    public ScalePlan PlanFor(int origW, int origH, int monLeft, int monTop)
    {
        double ratio = (double)origW / origH;
        var pick = ScalingTargets.All
            .FirstOrDefault(t => Math.Abs((double)t.Width / t.Height - ratio) < 0.02
                                 && t.Width < origW);

        if (pick is null)
            return new ScalePlan(origW, origH, origW, origH, 1.0, 1.0, monLeft, monT

        return new ScalePlan(origW, origH, pick.Width, pick.Height,
                             (double)pick.Width  / origW,
                             (double)pick.Height / origH,
                             monLeft, monTop);
    }

    // Screen pixel → model pixel (used when reporting cursor_position back)
    public (int x, int y) ScreenToModel(ScalePlan p, int sx, int sy) =>
        ((int)Math.Round((sx - p.MonitorLeft) * p.FactorX),
         (int)Math.Round((sy - p.MonitorTop) * p.FactorY));

    // Model pixel → screen (virtual-desktop) pixel — used for mouse_move/click
    public (int x, int y) ModelToScreen(ScalePlan p, int mx, int my)
    {
        if (mx < 0 || mx > p.ScaledWidth || my < 0 || my > p.ScaledHeight)
            throw new ArgumentOutOfRangeException(
                $"Model coordinates ({mx},{my}) outside scaled bounds {p.ScaledWidth
```

```
        return ((int)Math.Round(mx / p.FactorX) + p.MonitorLeft,
                (int)Math.Round(my / p.FactorY) + p.MonitorTop);
    }
}
```

Apply during capture using ImageSharp:

csharp

```
public ScreenshotResult Screenshot(int monitorIndex, string savePath,
                                   bool downscale, bool grayscale, string format)
{
    var raw = _capture.CaptureMonitorPng(monitorIndex, out var w, out var h,
                                         out var left, out var top);
    using var img = Image.Load<Rgba32>(raw);

    var plan = _mapper.PlanFor(w, h, left, top);
    if (downscale && (plan.ScaledWidth != w || plan.ScaledHeight != h))
        img.Mutate(ctx => ctx.Resize(plan.ScaledWidth, plan.ScaledHeight, KnownResam

    if (grayscale)
        img.Mutate(ctx => ctx.Grayscale(GrayscaleMode.Bt709));

    img.Save(savePath, new PngEncoder { CompressionLevel = PngCompressionLevel.BestC

    _activePlanByMonitor[monitorIndex] = plan;   // remember so mouse calls can reve
    return new ScreenshotResult(savePath, w, h, plan.ScaledWidth, plan.ScaledHeight,
                                plan.FactorX, plan.FactorY, plan.MonitorLeft, plan.M
}
```

Format choice: **PNG is the right default** for vision models — Anthropic's reference saves PNG, screenshots are graphics with sharp edges and text where JPEG artifacts hurt OCR. Use JPEG only when bandwidth dominates (e.g., over remote stdio); a Q=82 JPEG of a 1024×768 grayscale screenshot is ~50 KB vs ~150-300 KB for PNG. Grayscale cuts file size further by ~50-60% and is fine for layout reasoning, but degrades small colored UI accents — make it a per-call boolean.

G. Input — SendInput patterns

Mouse-move using virtual desktop normalization:

csharp

```
public void MouseMoveScreen(int sx, int sy)
{
    int vsLeft   = Win32.GetSystemMetrics(Win32.SM_XVIRTUALSCREEN);
    int vsTop    = Win32.GetSystemMetrics(Win32.SM_YVIRTUALSCREEN);
    int vsWidth  = Win32.GetSystemMetrics(Win32.SM_CXVIRTUALSCREEN);
    int vsHeight = Win32.GetSystemMetrics(Win32.SM_CYVIRTUALSCREEN);

    int nx = (int)Math.Round((sx - vsLeft) * 65535.0 / (vsWidth  - 1));
    int ny = (int)Math.Round((sy - vsTop)  * 65535.0 / (vsHeight - 1));

    var inputs = new INPUT[]
    {
        new() { type = 0 /*MOUSE*/, U = new InputUnion { mi = new MOUSEINPUT {
            dx = nx, dy = ny,
            dwFlags = 0x0001 /*MOVE*/ | 0x8000 /*ABSOLUTE*/ | 0x4000 /*VIRTUALDESK*/
        }}}
    };
    Win32.SendInput((uint)inputs.Length, inputs, Marshal.SizeOf<INPUT>());
}
```

Note `vsWidth - 1` and `vsHeight - 1` — this is the standard off-by-one fix discussed in libuiohook issue #21.

Unicode typing (no keyboard-layout sensitivity):

csharp

```
public void TypeText(string text, int delayMs)
{
    foreach (var rune in text.EnumerateRunes())
    foreach (var unit in rune.ToString())     // surrogate pairs become two inputs
    {
        var down = new INPUT { type = 1 /*KEYBOARD*/, U = new InputUnion { ki = new
            wScan = unit, dwFlags = 0x04 /*UNICODE*/ }};
        var up   = down;
        up.U.ki.dwFlags = 0x04 | 0x02 /*UNICODE | KEYUP*/;
        var arr = new[] { down, up };
        Win32.SendInput(2, arr, Marshal.SizeOf<INPUT>());
        if (delayMs > 0) Thread.Sleep(delayMs);
    }
}
```

Hotkeys: press all VKs down in order, release in reverse. For scroll, set `mouseData = clicks * 120 /*WHEEL_DELTA*/` and `dwFlags = MOUSEEVENTF_WHEEL` (or `MOUSEEVENTF_HWHEEL` for horizontal).

1. `WithToolsFromAssembly()` **emits IL2026.** Replace with explicit `.WithTools<T>()` per tool class (csharp-sdk issue #317).
2. **Tool method return types must be in your** `JsonSerializerContext`**.** Any record/class you return needs `[JsonSerializable]`. Missing types throw at runtime: *"The JSON value could not be converted"* (csharp-sdk issue #1053). GitHub
3. `Marshal.SizeOf<T>()` **works under AOT** for blittable structs — it does not require reflection at runtime for those.
4. `EnumDisplayMonitors` **callback** is a delegate. Under AOT delegates work, but if you store the delegate field in a struct or pass it across managed-unmanaged boundaries repeatedly, keep a managed reference alive (a `GC.KeepAlive(callback)` at the end of the call is sufficient) — without this you can occasionally see access violations.
5. `InvariantGlobalization=true` halves the binary and is fine for a Win-only server; if you want full Unicode collation use ICU and accept ~3-4 MB extra.
6. **ImageSharp** `Configuration.Default` roots every codec — for the smallest binary, instantiate a custom `Configuration` and add only PNG: `new Configuration(new PngConfigurationModule())` then call `image.Save(stream, new PngEncoder())`. This is the workaround discussed in ImageSharp issue #2486 and shipped in 3.1.0.
7. **stdio transport requirement**: never write to stdout. The Microsoft .NET Blog example explicitly demonstrates `options.LogToStandardErrorThreshold = LogLevel.Trace`. `AddMcpServer` with the generic host removes the default console logger. Codersera

I. Token-cost arithmetic to justify the downscale

Anthropic's Vision documentation states: *"An image uses approximately width * height / 750 tokens, where the width and height are expressed in pixels."* (platform.claude.com/docs/en/build-with-claude/vision). It also publishes per-model caps and automatic downscaling: non-Opus-4.7 models cap at 1,568 tokens (effectively a 1568px long edge), and Opus 4.7 caps at 4,784 tokens (2576px long edge). claude claude

Concrete examples:

- Native 4K (3840×2160) → raw 3840×2160/750 ≈ **11,059 tokens**, but the API automatically downscales to fit the model cap (~1,568 tokens for Sonnet, ~4,784 for Opus 4.7) — at lower model accuracy than if you'd downscaled yourself.
- WXGA (1280×800) → 1280×800/750 ≈ **1,365 tokens** (the Vision docs' reference table shows 1000×1000 ≈ 1,334 tokens as the comparable anchor). claude
- XGA (1024×768) → 1024×768/750 ≈ **1,049 tokens**.

So pre-downscaling a 4K screen to WXGA is **roughly an 8× input-token reduction** *and* yields more accurate clicks, because Anthropic's own image-resize step (when you exceed limits) is what they warn results in "lower model accuracy and slower performance than implementing scaling in your tools directly." GitHub

Recommendations

Stage 1 — Skeleton + Monitor enumeration + Screenshot (3-4 days).

- Create the project with `<PublishAot>true</PublishAot>`, the manifest above, `ModelContextProtocol` 1.3.0 and `SixLabors.ImageSharp` 4.0.0 packages.
- Implement `MonitorRegistry`, `CoordinateMapper`, and the `list_monitors` + `screenshot` tools.
- Verify `dotnet publish -c Release -r win-x64` produces a single `.exe` under 20 MB, and that the screenshot tool round-trips a PNG correctly on a multi-monitor setup with DPI scaling.
- **Threshold to move on:** screenshot of a non-primary monitor at 150% DPI scale produces a 1280×800 PNG whose pixel content visually matches that monitor.

Stage 2 — Mouse + Keyboard with coordinate remap (2-3 days).

- Add `mouse_move`, `mouse_click`, `mouse_drag`, `mouse_scroll`, `key_press`, `key_hotkey`, `type_text`, `wait`, `cursor_position`.
- Every mouse tool takes `coord_space` defaulting to `model`; convert via `ModelToScreen` using the most recent `ScalePlan` for that monitor.
- Manual test: have Claude (or any MCP host) call `screenshot`, then click coordinates the model identifies — verify clicks land in the expected place on the chosen monitor.
- **Threshold to move on:** end-to-end "open Notepad on monitor 1, type 'hello', save as C:\tmp\hi.txt" passes from a Claude Desktop session.

Stage 3 — File ops + launch + shell (1 day).

- `read_file`, `write_file`, `create_folder`, `launch_app`. Keep them dumb wrappers over `System.IO` / `Process.Start`.

Stage 4 — Optimize and harden (2-3 days).

- Switch ImageSharp to an explicit `Configuration` with only PNG to reduce binary size.
- Add `--monitor` and `--max-resolution` CLI flags so the host can pick defaults.
- Add a one-line "visual flash" overlay (à la Windows-MCP) so the user can see what the agent is doing — useful debugging aid.
- Add structured logs (stderr-only) with `Microsoft.Extensions.Logging`.

Recommended escalation triggers (signals to change the plan):

- *If clicks land 2–4 px off on a high-DPI laptop:* you forgot the `PerMonitorV2` manifest — without it, GDI returns virtualized coordinates and clicks land at virtualized positions.

- *If BitBlt returns black for a particular app (Chrome with HW accel, some Electron apps):* migrate that one path to `Windows.Graphics.Capture` via CsWinRT, keep BitBlt as default.
- *If AOT publish warnings exceed ~20 IL2026 warnings:* most will be in ImageSharp; suppress with `<NoWarn>IL2026;IL3050</NoWarn>` only after auditing each one. Don't blanket-suppress in your own code.
- *If you ever need browser-DOM-aware tools:* don't build them — install Windows-MCP alongside and use both servers from the same client.

Don't bother with sandboxing, OAuth, video recording, or accessibility-tree (`State-Tool`/UIA) scraping for v1. The user explicitly excluded sandboxing; vision-only operation matches Anthropic's design.

Caveats

- **`computer_20251124` adds a `zoom` action** that returns a sub-region at full resolution; this is not yet implemented in `claude-quickstarts` and Anthropic notes that *"Claude Opus 4.7 supports up to 2576 pixels on the long edge, and its coordinates are 1:1 with image pixels (no scale-factor conversion required)"* — meaning for the newest Opus models you may want to opt out of downscaling entirely. Make `downscale` a per-call boolean and surface the chosen target name in the tool result so the model knows the coordinate space. Anthropic
- **ImageSharp does not declare `IsAotCompatible=true`** in its csproj as of v4.0.0 (May 12, 2026). It publishes and runs correctly under PublishAot per the closure of issue #2486, but you will see trim analyzer warnings. The library's `AotCompilerTools` class is for Mono/Xamarin iOS AOT — not relevant to CoreCLR PublishAot but easy to confuse. GitHub
- **`System.Drawing.Common` does work on Windows under AOT** (it's now Windows-only and ships in the .NET 10 NuGet at v10.0.7), but Microsoft has signaled it will only evolve for Windows Forms/GDI+ scenarios. New code should not adopt it. NuGet
- **`MOUSEEVENTF_VIRTUALDESK`'s 0-65535 mapping is integer-quantized** across the entire virtual desktop. On a 7680px-wide multi-monitor setup, one quantization unit is ~0.12 px — fine for vision-targeted clicks, but if you need pixel-perfect drag drops use `MOUSEEVENTF_MOVE` (relative) sequences after `SetCursorPos`.
- **No production C#/.NET computer-use MCP server exists publicly** (as of May 2026) to copy from — `salty-flower/ModelContextProtocol.NET` is a competing SDK demonstrating AOT compatibility for tool integration, not a computer-use server. You are building something new in C#; the Python references (Windows-MCP, claude-quickstarts) are your behavioral specs. GitHub
- **Per-monitor DPI awareness can change physical pixel dimensions returned by GetMonitorInfo** depending on when you call `SetProcessDpiAwarenessContext`. Call it as the very first line in `Main` (or rely on the manifest) — calling it after any WinForms/WPF or even some `System.Drawing` initialization will leave the process in System-DPI mode, and your monitor bounds will be virtualized 96-DPI logical pixels rather than physical pixels.

- **Anthropic's downscale logic only scales down** (when `target.width < self.width`). If you're capturing a 1024×600 netbook display, the reference implementation does nothing — but the model is tuned for at-least-XGA imagery, so you may want to letterbox to 1024×768 (per the README's recommendation) rather than send sub-XGA.
- **The MCP C# SDK shipped its 1.0.0 stable release the week of February 23, 2026** and is currently at **v1.3.0 (May 8, 2026)**; the API surface is now stable but minor versions can still tighten serializer behavior. Pin the exact version and review the changelog before upgrading; ongoing work in csharp-sdk issue #795 is making every `With*` overload accept user-supplied `JsonSerializerOptions`, which will further improve AOT ergonomics. GitHub